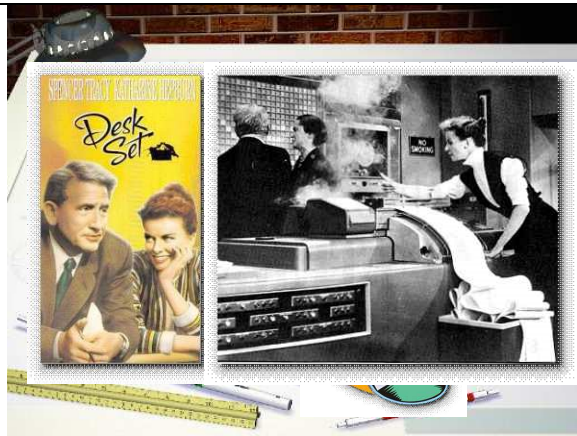




Slide 1

CS 170

Java Programming 1

True or False

Duration: 00:00:26

Advance mode: Auto

Notes:

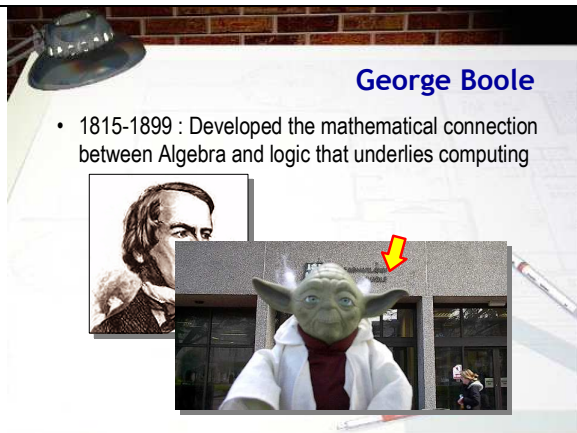
In the 1950s, where computers were first making inroads into society at large, Americans were inundated with media accounts of “mechanical brains” that could work “faster than a human, tirelessly, and without errors.” White-collar workers worried that their world would become automated, their jobs obsolete. In the 1957 movie named *Desk Set*, Spencer Tracy played Richard Sumner, an engineer hired to install a computer for Katherine Hepburn’s Bunny Watson, head of a television network’s research department. He assured her that the machine will help her become more productive, but Watson was justifiably suspicious: the computer fired not only her but also the entire research staff, the head of the network, and Sumner, who wasn’t even on the payroll.

So, why do we think of computers as “thinking machines?” That’s what we’ll be learning about in the next portion of CS 170.

Welcome to the CS 170, Java Programming 1 lecture on True or False. In this lesson, you’ll be introduced to the last of Java’s primitive types, the boolean. The boolean type is what allows computers to make decisions and tirelessly repeat any given task. Before we tackle selection and repetition, though, we’ll begin by looking at the boolean type itself and how you use boolean variables and boolean methods.

In this lecture, we’ll continue working with the TwoCards ACM Graphics program from the last lecture, so you can go ahead and open it up in BlueJ, along with this week’s lab-exercise document.

Ready, let’s get started.



Slide 2

George Boole

Duration: 00:00:36

Advance mode: Auto

Notes:

George Boole, is one the most famous mathematicians of the 19th century.

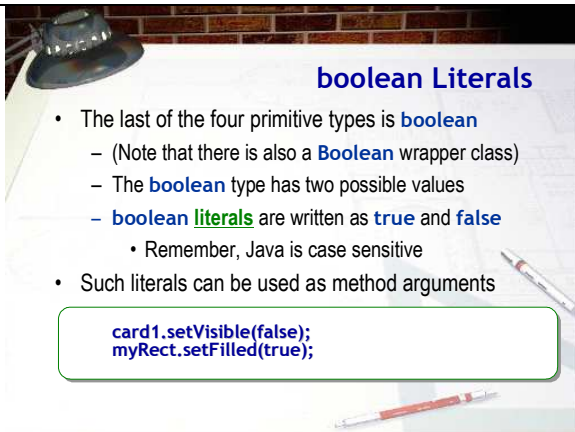
Boole was born in 1815 to a working-class family. He was a self-taught child prodigy in both science and Latin, who couldn’t afford to attend college, yet became a professor of Mathematics (and later Dean of Science) at Queens College in Cork Ireland.

Boole’s 1854 paper, “An Investigation Into the Laws of Thought, on Which are Founded the Mathematical Theories of Logic and Probability” detailed the relationship between algebra and logic, and it is this study on which much of today’s computer technology is based. The result is now known as Boolean logic or Boolean algebra.

Boole died on 8th December, 1864. One November day he walked to school in the pouring rain and proceeded to teach in wet clothes. As a result he contracted a chest infection which ultimately turned into pneumonia.

His condition was made worse by his wife's attempts to help him. She subscribed to the widely held view that the cure should match the cause and so while Boole took to his bed, she threw buckets of cold water over him.

Finally, here's a rare picture of one of Boole's most famous later disciples, on a visit to the Boole library in Cork Ireland.



boolean Literals

- The last of the four primitive types is **boolean**
 - (Note that there is also a **Boolean** wrapper class)
 - The **boolean** type has two possible values
 - **boolean literals** are written as **true** and **false**
 - Remember, Java is case sensitive
- Such literals can be used as method arguments

```
card1.setVisible(false);  
myRect.setFilled(true);
```

Slide 3

boolean Literals

Duration: 00:00:52

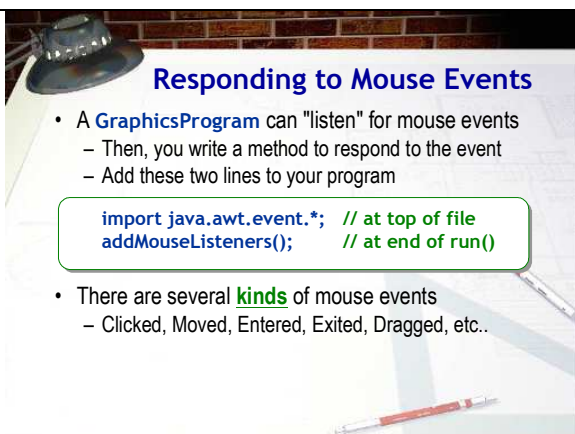
Advance mode: Auto

Notes:

Because the boolean type was named after George Boole when used as a generic term, it's always capitalized. The Java primitive type boolean, though, is all lower-case. Just like with the other primitives, Java has a wrapper class, Boolean with a capital B, that contains methods for converting a boolean to a String and a String to a boolean, as well as other constants and methods useful when dealing with boolean primitives.

The boolean type has two possible values, true and false. These literal values are written in all lowercase. The most common use for boolean literals is as method parameters. You can use boolean literals to make a GImage object visible or invisible using the setVisible() method, or to specify whether a GRect object should have a background or not by using the setFilled() method.

Now that you know how to use boolean literals, put them to work manipulating the cards in the TwoCards program.



Responding to Mouse Events

- A **GraphicsProgram** can "listen" for mouse events
 - Then, you write a method to respond to the event
 - Add these two lines to your program

```
import java.awt.event.*; // at top of file  
addMouseListeners();   // at end of run()
```

- There are several **kinds** of mouse events
 - Clicked, Moved, Entered, Exited, Dragged, etc..

Slide 4

Responding to Mouse Events

Duration: 00:00:49

Advance mode: Auto1

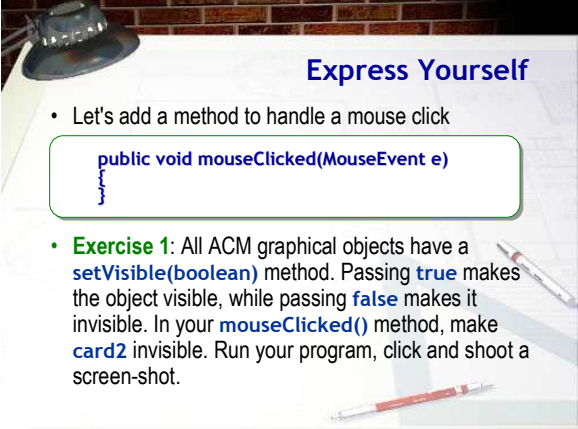
Notes:

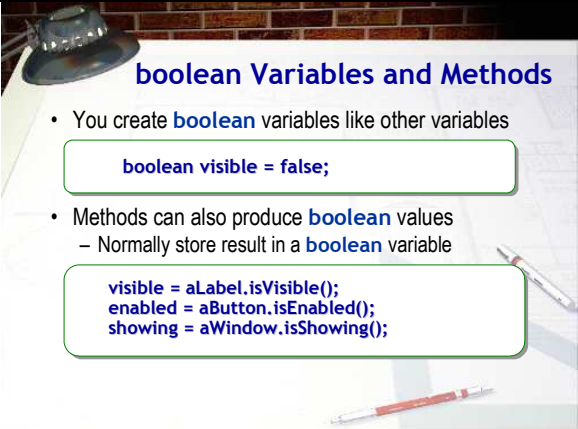
What we'd like to do is to be able to drag, flip or discard the cards on the table by clicking on them. To do that, we need to write some code in response to a mouse click. You'll learn all the details about how this works in Chapters 9 and 10, which we'll cover a little later in the semester. For right now, though, we'll use the simplified event handling provided by the ACM GraphicsProgram class. For those of you who don't want to learn two different ways to handle events, don't worry. The ACM classes automated some of the steps required to respond to events, but the steps that remain are the same ones you'll need when you move up to Java's built-in "real" event handling.

Once we've set up the program to listen for mouse events, you'll write a method to respond to the event. To set up the program, you need to add these two lines to your program:

- At the very top of the file, add import java.awt.event.*; Now I know that you already have java.awt.* which imports all of the classes in Java's AWT package. However, that doesn't import subpackages like the event package, so you need the second import statement.
- Second, at the end of the run() method, add the statement

	<code>addMouseListeners()</code>. This is the ACM method that hides several of the steps that you'd normally have to do manually. Once everything is set up, you can respond to all different kinds of mouse events: whether the mouse has been clicked, whether it's been moved, whether it's been dragged. Whether the mouse has entered or exited the screen territory of a particular component, and so on. For each one of these events, you simply write a different method. Let's just start with the method for a simple click.
--	--

 Express Yourself - Let's add a method to handle a mouse click <pre>public void mouseClicked(MouseEvent e)</pre> Exercise 1: All ACM graphical objects have a <code>setVisible(boolean)</code> method. Passing <code>true</code> makes the object visible, while passing <code>false</code> makes it invisible. In your <code>mouseClicked()</code> method, make <code>card2</code> invisible. Run your program, click and shoot a screen-shot. Slide 5 Express Yourself Duration: 00:00:25 Advance mode: Auto	Notes: To respond to mouse clicks, add a method to your program that has this signature. Notice that the method is named <code>mouseClicked</code> (starting with a lowercase m) and it takes one single parameter which is a <code>MouseEvent</code>. When you type in the word <code>MouseEvent</code>, you must use a capital M and a capital E, just as I've shown here.) Once you've got that typed in, you'll add statements to the method body. Then, any time someone clicks the mouse anywhere in your program, those statements will be executed. Let's start by adding a single statement that calls a method using a boolean parameter. For Exercise 1, use the <code>setVisible()</code> method to make <code>card2</code> invisible. All of the ACM objects have a <code>setVisible()</code> method. If you pass <code>true</code>, the object becomes visible; if you pass <code>false</code>, it disappears. When you're done, compile and run your program. Click the screen and make sure that the face down card (<code>card2</code>) disappears. Start a new section in your exercise document and then shoot me a screen-shot of the result and paste it in as Exercise 1.
--	--

 boolean Variables and Methods - You create <code>boolean</code> variables like other variables <pre>boolean visible = false;</pre> - Methods can also produce <code>boolean</code> values - Normally store result in a <code>boolean</code> variable <pre>visible = aLabel.isVisible(); enabled = aButton.isEnabled(); showing = aWindow.isShowing();</pre> Slide 6 boolean Variables and Methods Duration: 00:00:49 Advance mode: Auto	Notes: You create a boolean variable the same way you create an int, long, or double variable: you start with the type, select a name, and then (optionally) initialize your variable. Here, for instance, is a boolean variable named <code>visible</code>, initialized to the literal value <code>false</code>. Most students easily understand methods like <code>Math.Random()</code>, or <code>Math.pow()</code> which return the results of a numeric calculation. We can also have methods that return boolean, that is, true or false, values. These boolean methods are often also known as predicate methods. You'll be writing some predicate methods as part of your homework this week. When you call a boolean method you'll often use a boolean variable to store the result that the method returns. Here are some examples using a few boolean methods in the AWT: we could ask a <code>GLabel</code> object if it was visible and store the
--	--

result in the visible variable.

- we could ask a Button object if it was enabled or disabled and store the results in a boolean variable enabled.
- we could ask a Window object if it was showing on-screen (which is a little different than being visible) and store the result like this.

The boolean NOT Operator

- The unary **NOT** symbol is the exclamation point !

! Boolean Operand → Boolean result

- It appears in front of its **boolean** operand
- It produces the reverse of the expression it operates on

```
boolean visible = true;
boolean what = !visible; // false
visible = !visible;     // toggle
```

Slide 7

The boolean NOT Operator

Duration: 00:00:44

Advance mode: Auto

Notes:

Just like we have boolean literals, boolean variables, boolean methods, we also have boolean operators. Just like a numeric operator, say addition, performs a calculation with numeric operands, a boolean operator performs calculations with boolean values or variables.

So, what kind of calculations can we perform? Logical calculations, of course!

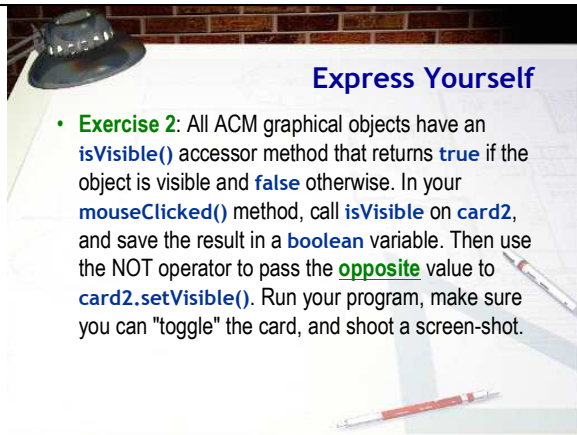
The first boolean operator we'll look at is called the NOT operator. This is a unary operator. Remember that unary operators perform their work on a single operand. The symbol for the NOT operator is the exclamation point and it appears immediately in front of its operand.

Just like other operators, after it's finished doing its work, the NOT operator produces a value—in this case a new boolean value. The new boolean value will be the opposite of whatever value the operand contains: if the operand is true, then not true is false; if the operand is false, then not false is true.

Here are some examples:

- The boolean variable visible is initialized with the value true.
- The boolean value what is then initialized with the expression NOT visible. Since visible was true, NOT visible is false, and so that's the value stored in what.

One really common operation with the not operator is to switch a variable between true and false and then back again. This is called a toggle. Here's how we'd toggle the visible variable from true to false. If we applied exactly the same statement again, it would switch from false to true.



Express Yourself

- Exercise 2:** All ACM graphical objects have an `isVisible()` accessor method that returns `true` if the object is visible and `false` otherwise. In your `mouseClicked()` method, call `isVisible` on `card2`, and save the result in a `boolean` variable. Then use the NOT operator to pass the **opposite** value to `card2.setVisible()`. Run your program, make sure you can "toggle" the card, and shoot a screen-shot.

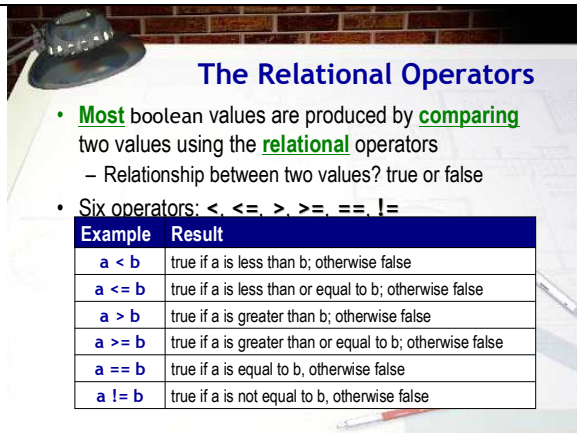
Slide 8
Express Yourself
 Duration: 00:00:11
 Advance mode: Auto

Notes:

OK, now it's your turn. In Exercise 1 you used the `setVisible()` method. All ACM graphical objects also have an accessor predicate method named `isVisible()`. As you might expect, `isVisible()` returns `true` if the object is visible, and `false` otherwise.

For Exercise 2, in your `mouseClicked()` method, call the `isVisible()` method on card 2 and save the result in a boolean variable. Then, use the NOT operator to pass the opposite value to `card2.setVisible()`, replacing the literal value you'd used before.

Compile and run your program and make sure clicking toggles the card, so that it appears, disappears and then reappears. Shoot me a screen-shot both visible and invisible.



The Relational Operators

- Most** boolean values are produced by **comparing** two values using the **relational** operators
 - Relationship between two values? true or false
- Six operators: `<`, `<=`, `>`, `>=`, `==`, `!=`

Example	Result
<code>a < b</code>	true if a is less than b; otherwise false
<code>a <= b</code>	true if a is less than or equal to b; otherwise false
<code>a > b</code>	true if a is greater than b; otherwise false
<code>a >= b</code>	true if a is greater than or equal to b; otherwise false
<code>a == b</code>	true if a is equal to b, otherwise false
<code>a != b</code>	true if a is not equal to b, otherwise false

Slide 9
The Relational Operators
 Duration: 00:00:29
 Advance mode: Auto

Notes:

Although boolean literals and boolean variables can be useful, most of the time, you'll create the Boolean values used by your program by making use of Java's relational or equality operators, often in the context of a selection or iterative conditional statement.

A relational operator compares two values and returns a boolean value based on a question about the relationship of one value to the other. (That's why they're called the relational operators.) Using a relational operator is a little like the parlor game, "Twenty questions." You use the operator to "ask" if one number is larger than, smaller than, or equal to, another. The operator "responds" with true or false.

Java has six operators which are used to compare numerical values: less-than, less-than or equal, greater-than, greater-than or equal, equal (which requires two equals signs) and not equal.

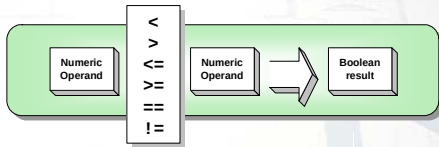
Here's an illustration of the use of each.

- `a < b` true if a is less than b; otherwise false
- `a <= b` true if a is less than or equal to b; otherwise false
- `a > b` true if a is greater than b; otherwise false
- `a >= b` true if a is greater than or equal to b; otherwise false
- `a == b` true if a is equal to b, otherwise false
- `a != b` true if a is not equal to b, otherwise false

Those operators that require two symbols (`<=`, `>=`, `==`, and `!=`) must be typed with no space between the two symbols.

Primitive Relations

- You can compare most primitive **numeric** types, including **char**, but not **boolean**, without problem
 - You don't have to worry about precision or size



Notes:

Each of the relational operators requires two primitive operands, and the expression produces a boolean (true/false) result. You can only do this with comparable primitive types; fortunately, most primitive numeric types are comparable, including the char type. You normally don't have to worry about mixed-type comparisons; for the most part, they will work just like you'd expect.

Slide 10

Primitive Relations

Duration: 00:00:19

Advance mode: Auto

Express Yourself

- Exercise 3:** Here are some primitive variables along with some relational expressions. Type these variables into Code Pad and then try out each of the expressions below. Shoot a screen-shot of the result

```
b > sh
c < a
f == a
f < a
c > a
```

```
// Variables
byte b = 2;
short sh = 3300;
double a = 2.34;
float f = 2.34F;
char c = 'A';
```

Variables of
several different
primitive types

Notes:

OK, it's your turn again. You don't need to close the TwoCards program, we'll return to it again shortly. For right now, though, let's experiment with some relational expressions. The results may hold a few surprises.

First, type these variable definitions into Code Pad. Note that we have byte, short, double, float and char variables.

Next, type these expressions in Code Pad. Remember that we want to see the values, so just hit enter after typing the expression; don't add a semi-colon.

Shoot me a screen-shot of the result.

As you can see, you can compare bytes with shorts, characters with doubles and floats and doubles. One thing might be confusing to you though. Why aren't the values in the double a and the float f, equal?

Let's take a quick look at that.

Slide 11

Express Yourself

Duration: 00:00:19

Advance mode: Auto

Floating-point Relations

- Generally, you **shouldn't use** == or != with reals
- Problems with **mixed type** comparisons
 - With mixed type, **2.34** and **2.34F** are NOT equal
 - They have different bit patterns

```
public static void main(String[] args)
{
    // 1. Comparing different types
    float a = 2.34F;
    double b = 2.34;

    System.out.println("a == b is " + (a == b));
    System.out.println("a > b is " + (a > b));
    System.out.println("a < b is " + (a < b));
}
```

Notes:

Even though you can compare numbers without regard for their types, that doesn't mean that the results will always be meaningful. As a general rule, you should never directly compare two floating point numbers using the equality operators (==, !=) because of the "round-off" errors that can occur.

Here's a short console-mode program that illustrates one of the pitfalls that can arise when comparing floating-point numbers. Let's look at a couple of instances:

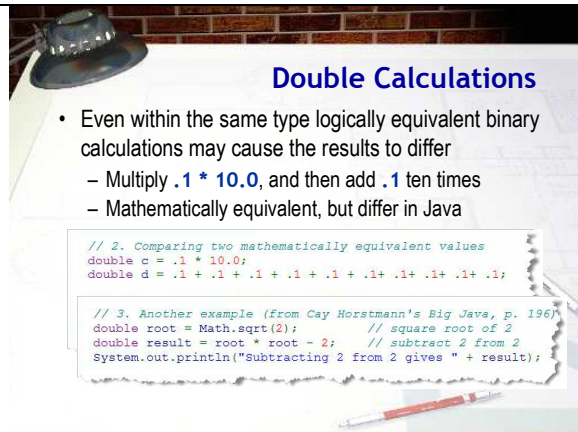
In section 1, two floating-point variables (a and b) are created and initialized using the values 2.34 and 2.34F. Then, those values are compared using the relational operators. If you type this in and run the code, you'll see that the two values are

Slide 12

Floating-point Relations

Duration: 00:00:31
Advance mode: Auto

never equal, because they have different bit patterns that can't be accurately compared. Now that's not always the case. Many times, if a number is an even power of two, for instance, the comparison will be correct, but often it won't, so you can rely on it.



Double Calculations

- Even within the same type logically equivalent binary calculations may cause the results to differ
 - Multiply `.1 * 10.0`, and then add `.1` ten times
 - Mathematically equivalent, but differ in Java

```
// 2. Comparing two mathematically equivalent values
double c = .1 * 10.0;
double d = .1 + .1 + .1 + .1 + .1 + .1 + .1 + .1 + .1 + .1;

// 3. Another example (from Cay Horstmann's Big Java, p. 196)
double root = Math.sqrt(2); // square root of 2
double result = root * root - 2; // subtract 2 from 2
System.out.println("Subtracting 2 from 2 gives " + result);
```

Slide 13

Double Calculations

Duration: 00:00:31
Advance mode: Auto

Notes:

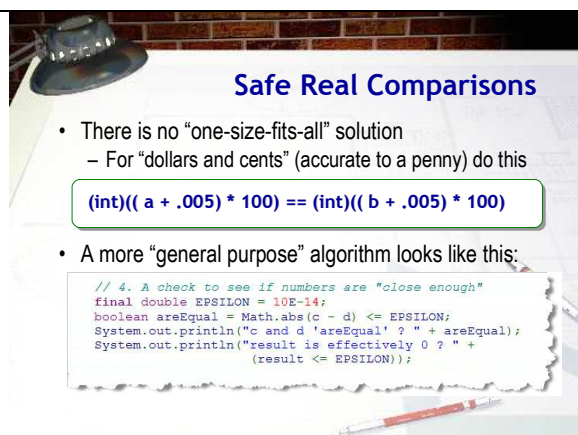
Calculations involving floating-point numbers are done in binary, so even if all operands have the same type, you cannot be sure that the bit-patterns will match exactly. Here are two examples.

Look at the variable definition for `c` which multiplies `.1` times `10`, yielding (theoretically) `1.0`. Note then the comparison of `c` with `1.0` on line 25. Is it true or false? I'm sure you realize that this should be true, and that's what Java reports. Well, if that's so, what about the variable `d`? Obviously, adding `.1` ten times is mathematically the same expression as multiplying `.1` times `10`, yet Java considers this one to be false.

Here's a second example your textbook. The double variable `root` is initialized using the `Math.sqrt()` function to get the square root of `2`. The variable `root` is multiplied by itself, (yielding, theoretically, `2`), and the literal value `2` is subtracted and the whole result is stored in the variable `result`. This result should be, mathematically, `0`. Instead, Java reports that the answer is `4.440892098500626E-16`.

Programmers encountering this behavior for the first time, often think that this is a bug, but it is not. This is not a bug in Java, or its ability to add. This is just the way that binary floating-point numbers work. It's in the nature of the numbers themselves.

The conclusion? Don't compare floating point numbers using the equality operators.



Safe Real Comparisons

- There is no "one-size-fits-all" solution
 - For "dollars and cents" (accurate to a penny) do this

```
(int)((a + .005) * 100) == (int)((b + .005) * 100)
```

- A more "general purpose" algorithm looks like this:

```
// 4. A check to see if numbers are "close enough"
final double EPSILON = 10E-14;
boolean areEqual = Math.abs(c - d) <= EPSILON;
System.out.println("c and d 'areEqual' ? " + areEqual);
System.out.println("result is effectively 0 ? " +
    (result <= EPSILON));
```

Slide 14

Notes:

To compare floating point numbers after a calculation, you'll have to first decide what is "close enough". If you buy gasoline, for instance, you might notice the price often priced to include 9/10 of a cent, but no one ever pays 9/10 of a cent—because our currency doesn't have amounts less than a penny. So, if you were comparing two floating-point numbers where it was important to be accurate only to a penny, you'd want the number `1.256` and `1.257` to be "equal". Here's a rough calculation that will do that.

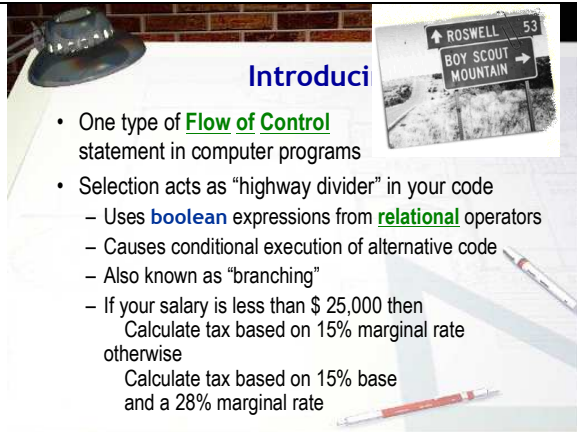
A more general purpose algorithm looks like this. Since type `double` is accurate to about 16 digits of precision, we can generalize by using a fudge factor (often called `EPSILON`) of

Safe Real Comparisons

Duration: 00:00:36

Advance mode: Auto

10E-14. Using this scheme, two double values are equal if the absolute value of their difference is less than or equal to EPSILON). When you run this, you'll see that the variables `c` and `d` are effectively equal, and that the variable `result` is effectively 0. This doesn't work perfectly if the numbers are very large—in the millions—in which case you may need to make your EPSILON factor larger.



Introducing Selection

- One type of **Flow of Control** statement in computer programs
- Selection acts as "highway divider" in your code
 - Uses **boolean** expressions from **relational** operators
 - Causes conditional execution of alternative code
 - Also known as "branching"
 - If your salary is less than \$ 25,000 then
 - Calculate tax based on 15% marginal rateotherwise
 - Calculate tax based on 15% base and a 28% marginal rate

Slide 15

Introducing Selection

Duration: 00:00:43

Advance mode: Auto

Notes:

The relational operators, and the boolean values that they produce, are important because they allow us to implement selection. Selection acts like a "highway divider" in your code; one minute you're on a trip to Boy Scout Mountain, the next you're knee deep in smoking men, and running around the desert with Gillian Anderson.

With selection, a relational operator is normally used to test two values. This test is called a condition. If the result of the test is true, then a particular section of code is executed; if the condition is false, then that section of code is skipped, and, perhaps, an alternative section of code is executed. Selection is also called branching, because any time you run the program you may take a different path through the code.

Here is an example of a selection statement using English like pseudocode:

If your salary is less than \$ 25,000 then

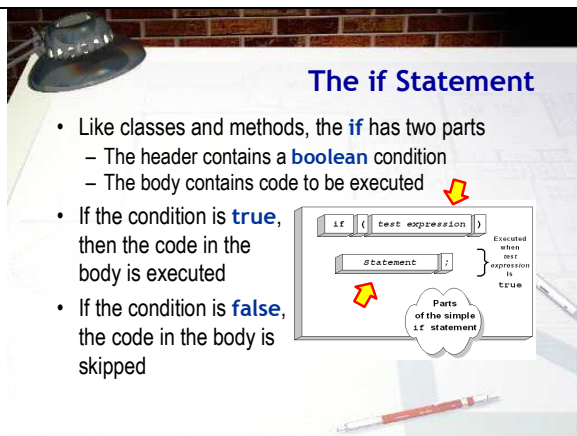
 Calculate tax based on 15% marginal rate

otherwise

 Calculate tax based on 15% base rate

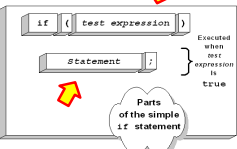
 and a 28% marginal rate

Java has many different kinds of selection statements which we'll cover next week. Today, though, let's finish up this week's material by looking at the if statement.



The if Statement

- Like classes and methods, the **if** has two parts
 - The header contains a **boolean** condition
 - The body contains code to be executed
- If the condition is **true**, then the code in the body is executed
- If the condition is **false**, the code in the body is skipped



Slide 16

Notes:

The if statement is the first, and simplest, branching instruction that we'll cover. Like classes and methods, the if statement consists of two parts: a header and a body. Here's an illustration below shows the syntax of the if statement.

With the if statement, the "header" is a boolean condition to be tested. The body of the if statement contains code to be executed.

Each if statement executes a single section of code after testing its boolean condition. If the condition is true, then the code is executed; if the condition is false, the code is skipped.

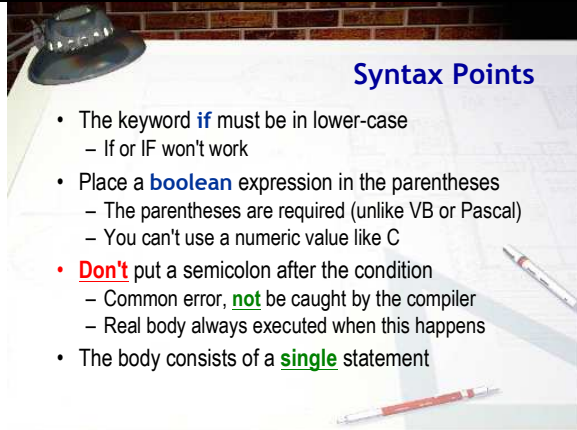
Unlike a method body, the code contained in the body of an if

The if Statement

Duration: 00:00:37

Advance mode: Auto

statement does not need to be enclosed in braces. If you do not use braces, however, only a single statement will be executed.



- The keyword **if** must be in lower-case
 - If or IF won't work
- Place a **boolean** expression in the parentheses
 - The parentheses are required (unlike VB or Pascal)
 - You can't use a numeric value like C
- **Don't** put a semicolon after the condition
 - Common error, **not** be caught by the compiler
 - Real body always executed when this happens
- The body consists of a **single** statement

Slide 17

Syntax Points

Duration: 00:00:59

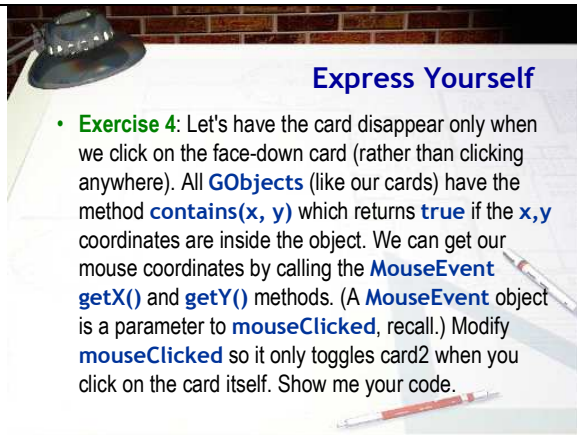
Advance mode: Auto

Notes:

Here are the important points to understand about the if statement syntax:

1. The if statement begins with the keyword if in lowercase; you cannot use If or IF.
2. The condition following the if keyword must—when evaluated—produce a boolean value. That means you can use boolean literals, (although you really shouldn't), boolean variables, or, (most likely), a relational expression.
3. The parentheses are required, unlike in Pascal or Visual Basic.
4. You can't use a numeric value, like you can in C or C++. Boolean types are not integers or convertible to integers in Java.
5. Make sure you don't put a semicolon after the condition. When you do this, the body of the statement becomes what is known as a "null" statement. This is a common error that is NOT caught by the compiler: it is perfectly legal, but meaningless. If you put a semicolon after the condition, you are saying, "In the event that this condition is true, don't do anything."
6. The body of an if statement is a single statement, ending with a semicolon. The statement will be executed if the condition is true.

Let's finish up this lesson by doing a couple of exercises with TwoCards.



Express Yourself

- **Exercise 4:** Let's have the card disappear only when we click on the face-down card (rather than clicking anywhere). All **GObjects** (like our cards) have the method **contains(x, y)** which returns **true** if the **x, y** coordinates are inside the object. We can get our mouse coordinates by calling the **MouseEvent** **getX()** and **getY()** methods. (A **MouseEvent** object is a parameter to **mouseClicked**, recall.) Modify **mouseClicked** so it only toggles card2 when you click on the card itself. Show me your code.

Slide 18
Express Yourself
 Duration: 00:00:11
 Advance mode: Auto

Notes:

One problem with the TwoCards applet is that the card disappears and reappears whenever you click anywhere in the applet. Let's change that and have the card disappear **ONLY** when you click on the face-down card.

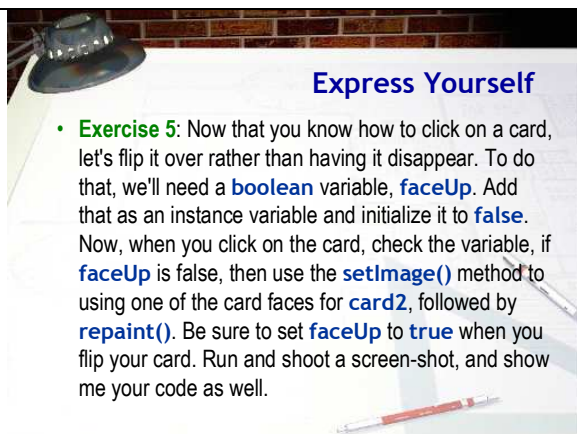
Here's how you do that:

1. All ACM graphical objects have a method named **contains()** that takes two int parameters, **x** and **y**. If the coordinates **x, y** are inside the object, then the method returns **true**.
2. Every **MouseEvent** object, like the parameter passed into the **mouseClicked()** method, has two methods, **getX()** and **getY()** that let you retrieve the location where the mouse was pointing when the click occurred.

To modify your **mouseClicked()** method:

1. First, retrieve the **x** and **y** location where the mouse was clicked and store it in two local int variables.
2. Use the **card2.contains()** method to see if the coordinates are inside **card2**. If they are, then toggle the card (that is, make it visible and invisible using the code that you've already written). If the coordinates aren't inside **card2** (that is, if you've clicked anywhere else on the program), then you shouldn't do anything.

A screen-shot wouldn't really tell me much, so for Exercise 4 just copy and paste the code you've written.



Express Yourself

- **Exercise 5:** Now that you know how to click on a card, let's flip it over rather than having it disappear. To do that, we'll need a **boolean** variable, **faceUp**. Add that as an instance variable and initialize it to **false**. Now, when you click on the card, check the variable, if **faceUp** is false, then use the **setImage()** method to using one of the card faces for **card2**, followed by **repaint()**. Be sure to set **faceUp** to **true** when you flip your card. Run and shoot a screen-shot, and show me your code as well.

Slide 19
Express Yourself
 Duration: 00:00:11
 Advance mode: Auto

Notes:

Now let's do something a little more ambitious. Instead of having the face-down card disappear and reappear, let's flip it over. What's a little tricky, though, is that we'll only want to flip it over if it's currently face down on the table.

Here's how to go about it:

1. First, create a boolean instance variable named **faceUp** and initialize it to false (since we'll deal our card face down).
2. Now, when you click on the card, check to see if the variable **faceUp** is true. If it is true, then just return (that is, you won't do anything.) Even though **mouseClicked()** is a void method, and doesn't need a return expression, you can still use the return statement (just **return;**) to end the function at any point.
3. In the body of the if statement left over from Exercise 4, comment out the code you used to toggle **card2** and add three more lines.

*The first line should use the **setImage()** method to display one of the card images. You'll pass **setImage()** a String just like you used for the **GImage** constructor.

	• *The second line should just be <code>repaint()</code>; This causes the program to notice that you've replaced the card face and to display the new version. • *Finally, the third line should set <code>faceUp</code> to <code>true</code>. This prevents you from clicking on a face-up card and getting a new value. That's it! Compile and run your program. Flip the second card and shoot me a screen-shot. Copy and paste your code as well.
--	--